

HPE CPP

Week-3 Report

Submitted by: Shubham Kumar

1. Analysis of QUIC Initial Packets

QUIC Initial packets use the Long Header format, exposing the most observable metadata of any QUIC packet type. Once the handshake completes, packets switch to the Short Header, significantly reducing visible fields. Initial packet analysis is therefore the primary opportunity for flow identification in an encrypted QUIC environment.

1.1 Observable Fields — Role and Use

Field	Size	Typical Value	Description	Use in Flow ID / Tracking
Header Form	1 bit	Always 1 (Long Header)	Confirms packet is a long-header type, indicating connection setup phase	Phase identification; separates handshake traffic from steady-state 1-RTT data
Fixed Bit	1 bit	Always 1 (all packets)	Present in every QUIC packet; designed for protocol ossification resistance	Reliable first-pass QUIC detector; any UDP pkt with bit=1 is a QUIC candidate
Long Pkt Type	2 bits	00=Initial 10=HS 11=Retry	Specifies the role of the long-header packet within the handshake lifecycle	Identifies Initial, Handshake, and Retry packets; enables per-stage counters
QUIC Version	32 bits	0x00000001 (v1)	Carried in every long-header packet; version 0x0 signals Version Negotiation	Protocol version inventory; VN packet detection; confirms QUIC v1 deployment
DCID	0-20 B	Random (client-chosen)	Primary connection identifier; stable across IP/port changes due to migration	Primary flow key; replaces the TCP 5-tuple for connection tracking

Field	Size	Typical Value	Description	Use in Flow ID / Tracking
SCID	0-20 B	Random (sender-chosen)	Sender's own CID; receiver uses it as DCID for reverse-direction packets	Bidirectional flow mapping; enables complete session reconstruction
Token Length	Var-int	0 = no token	Non-zero value indicates a Retry token is present, proving address ownership	Retry path detection; reveals anti-amplification enforcement by server
Packet Length	Var-int	Remaining bytes	Directly observable payload length; Initial pkts padded to ≥ 1200 B by RFC	Payload size estimation; path-MTU confirmation; burst sizing

1.2 Flow Identification Using Initial Packet Fields

A QUIC connection is tracked using a composite key derived from observable Initial packet fields:

- Primary key: Destination Connection ID (DCID) - persists across NAT rebinding and IP migration. All packets of a connection carry the same DCID, allowing the monitoring system to group them without relying on IP addresses.
- Secondary key: Source Connection ID (SCID)- enables bidirectional mapping by identifying the sender's own CID, which the remote endpoint will use as its DCID in reverse-direction packets.
- Fallback key: (src_ip, src_port, dst_ip, dst_port) - used when CID length is zero, or during the very first Initial packet before CIDs are negotiated.
- Version confirmation: QUIC Version field (0x00000001) confirms the packet belongs to QUIC v1. A value of 0x00000000 marks a Version Negotiation packet requiring special handling.
- Token presence: A non-zero Token Length confirms the client has received a Retry packet from this server and is completing address validation -an observable connection-level event.

1.3 Visible vs. Encrypted Header Fields

VISIBLE	ENCRYPTED / PROTECTED (Requires key material)
Header Form bit (1 bit)	Packet Number (header-protected)

VISIBLE	ENCRYPTED / PROTECTED (Requires key material)
Fixed Bit (1 bit)	Packet Number Length (header-protected)
Long Packet Type (2 bits)	Key Phase Bit (short header, protected)
QUIC Version (32 bits)	Reserved Bits (masked after protection)
DCID Length + DCID bytes	STREAM frames (stream ID, offset, data)
SCID Length + SCID bytes	ACK frames and flow control signals
Token Length + Token bytes	CRYPTO payload (TLS records)
Packet Length field	HTTP/3 headers, application payload

2. Inferring Streams from Traffic Patterns

The following analysis is grounded exclusively in values from dataset.csv. Each observation section states what the data shows; each interpretation section explains what it means for traffic classification.

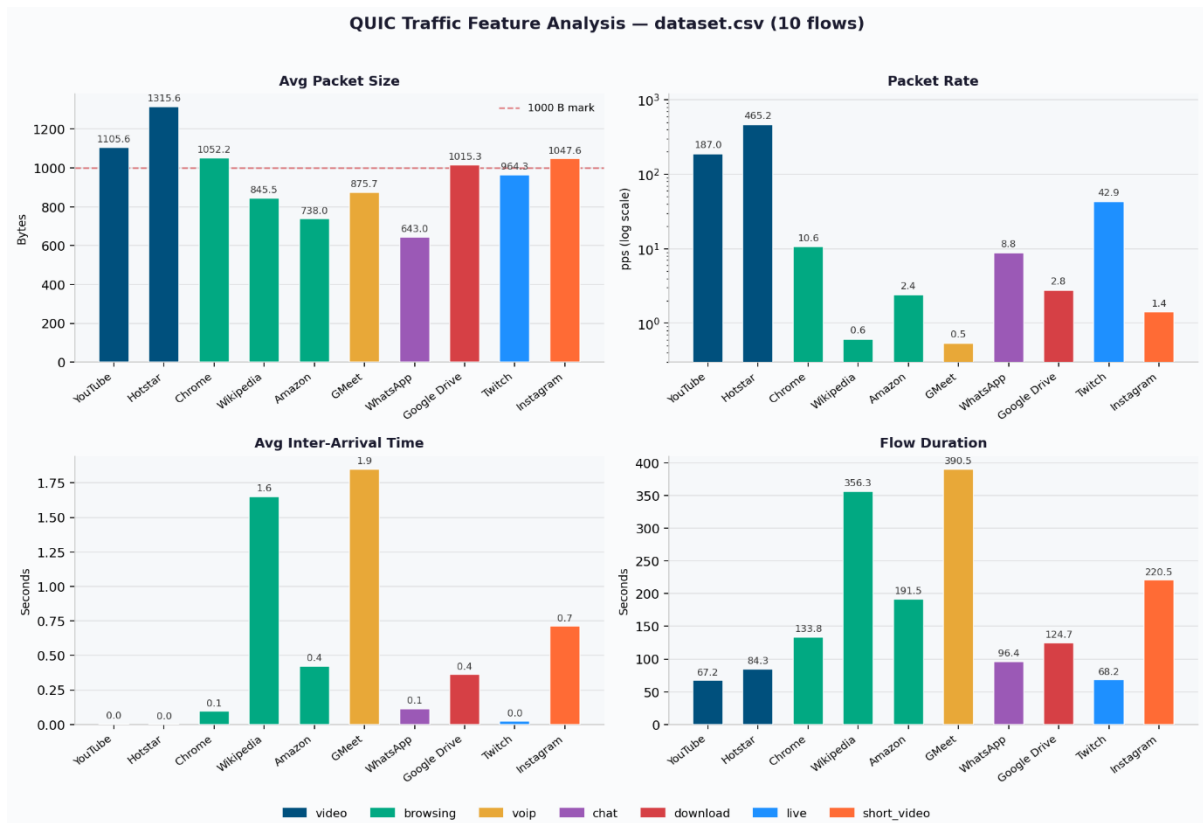
2.1 Feature-by-Feature Analysis

Feature	Observation (from dataset)	Interpretation
Avg Packet Size	Hotstar (1316 B) and YouTube (1106 B) are the two largest. WhatsApp (643 B) is the only flow below 700 B. Browsing flows span the widest range (738–1052 B) with the highest std_dev (avg 565 B across the three).	Large, consistent packet sizes indicate bulk media delivery (video, download). The unique low-size footprint of WhatsApp reliably identifies chat. High std_dev in browsing reflects the mix of small request headers and large HTML/resource responses.
Packet Rate	Hotstar (465 pps) and YouTube (187 pps) are separated from all other labels by more than an order of magnitude. Twitch (43 pps) is clearly distinct from non-streaming labels. GMeet (0.54 pps) and Wikipedia browsing (0.61 pps) are the lowest.	Packet rate is the strongest single discriminator in this dataset, separating video/live from all other types with a 4–10x gap. The GMeet capture represents an idle/background session, hence anomalously low rate for a VoIP label.

Feature	Observation (from dataset)	Interpretation
Avg Inter-Arrival Time	GMeet (1.851 s) and Wikipedia (1.650 s) have the highest IAT, nearly 2 s between packets on average. Hotstar (0.002 s) and YouTube (0.005 s) have near-zero IAT. Twitch (0.023 s) is low but above video. WhatsApp (0.114 s) is notably regular.	IAT is the inverse of rate and provides complementary evidence. Near-zero IAT confirms continuous media delivery. The regularity of WhatsApp IAT (vs. the high variance of browsing IAT) reflects chat protocol keepalives and message cadence rather than user-driven page loads.
Flow Duration	GMeet (390.5 s) is the longest by a significant margin. Wikipedia (356.3 s) and Instagram (220.5 s) are the next two. Video flows (YouTube 67.2 s, Hotstar 84.3 s) are the two shortest, alongside Twitch (68.2 s). These likely represent single-segment or short-session captures.	Long duration combined with low packet rate (GMeet, Wikipedia) reflects persistent sessions with sparse activity, a long reading session or idle conference call. Short duration with high rate (video, live) corresponds to intensive content fetch sessions rather than full-length streams.

2.2 Visual Analysis

Two graphs are included below. Figure 1 shows all four feature dimensions across all 10 flows. Figure 2 maps flows into a classification space using the two strongest discriminators.



2.3 Summary Comparison Table

Traffic Type	Pkt Size	Rate	Avg IAT	Duration	Distinguishing Pattern
video	1106–1316 B	187–465 pps	0.002–0.005 s	67–84 s	Highest rate by 10x; largest packets; near-continuous delivery; short capture window
live	964 B	42.9 pps	0.023 s	68 s	High rate but 10x lower than video; real-time segment push; similar short duration
browsing	738–1052 B	0.6–10.6 pps	0.09–1.65 s	134–356 s	Highest std_dev (565 B avg); most variable IAT; irregular user-driven request pattern
chat	643 B	8.78 pps	0.114 s	96 s	Only label below 700 B; moderate rate; semi-regular IAT reflecting message cadence
download	1015 B	2.78 pps	0.361 s	125 s	Large packets at low rate, sustained bulk transfer without bursting
voip	876 B	0.54 pps	1.851 s	391 s	Highest IAT and duration; lowest rate, background/idle meeting session capture
short_video	1048 B	1.41 pps	0.711 s	221 s	Large packets, low rate, moderate IAT, clip fetch-buffer-pause cycle pattern

3. Mapping Patterns to Encrypted Information

3.1 Pattern to Inference Table

The following rules are derived directly from dataset.csv values. Rules should be applied in the order listed; the first matching rule determines the label. All conditions use only observable metadata — no payload inspection is required.

Observable Metadata Condition	Inferred Label	Dataset Evidence
pkt_rate > 100 AND avg_pkt_size > 1100 B AND avg_iat < 0.01 s	video	YouTube: 187 pps, 1106 B. Hotstar: 465 pps, 1316 B. Sustained media segment delivery at near-MTU size; no other label meets all three conditions simultaneously.
pkt_rate 30-100 AND avg_iat < 0.03 s AND avg_pkt_size ~900-1000 B	live	Twitch: 43 pps, 964 B, 0.023 s IAT. Real-time stream at lower rate than pre-recorded video. Mid-rate cluster unique in the dataset.
avg_pkt_size < 700 B AND pkt_rate 5-15	chat	WhatsApp: 643 B, 8.78 pps. Only flow below 700 B. Small payload per message plus keepalive packets produces this unique small-packet, moderate-rate signature.
avg_pkt_size > 950 B AND pkt_rate < 5 AND avg_iat 0.3-0.5 s	download	Google Drive: 1015 B, 2.78 pps, 0.361 s IAT. Large packets at low rate, bulk sequential transfer without bursty rate spikes.
avg_pkt_size > 950 B AND pkt_rate < 3 AND duration > 180 s	short_video	Instagram: 1048 B, 1.41 pps, 221 s. Large packets, low rate, long duration, clip-level fetch followed by user pause cycle.
avg_iat > 1.5 s AND duration > 300 s AND pkt_rate < 1	voip	GMeet: 1.851 s, 391 s, 0.54 pps. Background/idle session. Long duration + sparse traffic is the defining signature; active VoIP would show different values.
std_dev > 550 B AND pkt_rate < 15 AND IAT variable (high variance)	browsing	Chrome/Wiki/Amazon: avg std_dev 565 B highest in dataset. Irregular IAT reflects user-driven page loads rather than protocol-driven transmission cadence.

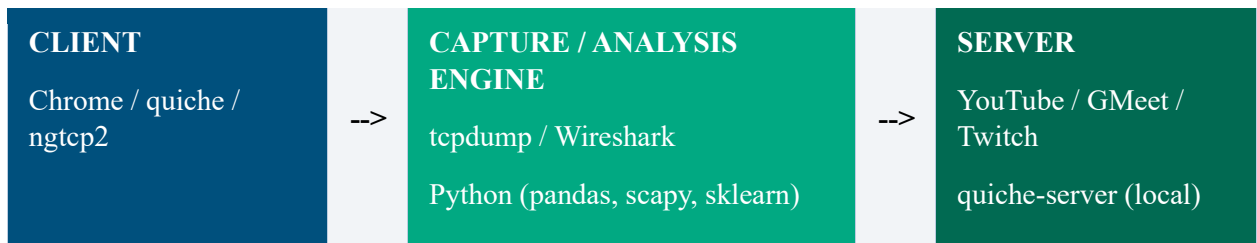
3.2 Why This Works Despite Encryption

QUIC encrypts stream identifiers, acknowledgments, flow control, and all application payload. However, the transport layer cannot hide the following observable side-channel properties:

- Throughput estimation: $\text{rate} \times \text{avg_pkt_size}$ estimates bandwidth without decryption. Hotstar: $465 \times 1316 \text{ B} = \sim 4.8 \text{ Mbps}$; YouTube: $187 \times 1106 \text{ B} = \sim 1.6 \text{ Mbps}$. Both consistent with their known video bitrates.
- IAT regularity reveals transmission model: applications with protocol-driven send schedules (media players, VoIP codecs) produce low IAT variance; user-driven applications (browsing, chat triggered by typing) produce high variance.
- Flow duration identifies session type: persistent low-rate flows (conferencing, long reads) vs. short high-intensity fetch sessions (video segments, file downloads).
- Packet size distribution (std_dev) exposes content mix: browsing produces the highest std_dev (avg 565 B) due to the simultaneous presence of small HTTP/3 request headers and large response objects within the same flow.
- Long-Short Header transition timestamps handshake completion: the shift from QUIC Long Header to Short Header packets on a given DCID is directly observable and marks the start of 1-RTT data exchange.
- Spin bit RTT estimation: the optional spin bit in 1-RTT Short Header packets toggles once per round-trip, allowing passive RTT measurement from observable packet headers alone.

4. Environment Setup for Testing and Validation

4.1 Network Topology



4.2 Analysis Pipeline



4.3 Tools and Components

Component	Tool	Purpose and Key Command
Traffic Generator	Chrome, quiche, ngtcp2	Generates realistic QUIC HTTP/3 flows. Chrome with enable-quic targets internet services; quiche/ngtcp2 enable local server testing.

Component	Tool	Purpose and Key Command
Packet Capture	Tcpdump	<code>sudo tcpdump -i eth0 -w capture.pcap 'udp port 443'</code> - captures all QUIC traffic to PCAP without decryption.
Live Analysis	Wireshark	QUIC-aware dissector decodes long/short headers, CIDs, version, and packet types. TLS keylog support for controlled lab decryption.
Field Extraction And Feature Aggregation	Python / pandas	Processes Wireshark-exported CSV using Python to compute flow-level features (avg_pkt_size, std_dev, avg_iat, duration, pkt_rate) from packet-level fields. Groups packets by DCID; computes avg_pkt_size, std_dev, avg_iat, duration, pkt_rate produces dataset.csv schema.
Packet Crafting	Python / Scapy	Custom QUIC-like packet generation for testing classification rules under controlled conditions.
Classification	Python / sklearn	Applies trained ML classifier; validates against ground-truth labels.
Testing & Verification	Python / sklearn	Validates classification accuracy using predicted vs actual labels (accuracy, confusion matrix).

4.4 Workflow Steps

- Generate traffic: Open Chrome with QUIC enabled and browse the target services (YouTube, GMeet, Twitch, etc.) for a defined capture window (60-400 s depending on flow type).
- Capture: `sudo tcpdump -i eth0 -w <service>.pcap 'udp port 443'` — one PCAP file per flow.
- Extract fields: Python / PyShark → iterates over packets in <file>.pcap and exports observable fields (timestamp, packet length, IPs, QUIC metadata where available) to packets.csv using CSV writer.
- Aggregate to flow features: Group by DCID; compute avg_pkt_size, std_dev, avg_iat, duration, pkt_rate per flow, this produces the dataset.csv schema exactly.
- Classify and Validate: Train a supervised ML model on extracted features (avg_pkt_size, std_dev, avg_iat, duration, pkt_rate) and predicts traffic labels for each flow; evaluates performance using accuracy and confusion matrix, flags low-confidence predictions as "unknown" for manual review